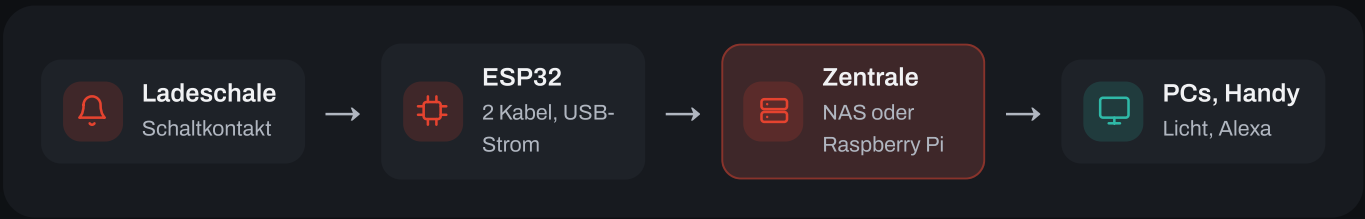
Heimalarm mit ESP32

Ein ESP32 liest die Ladeschale direkt aus – unter einer Sekunde, ohne Cloud. Zentrale ist eine UGREEN NAS oder ein Raspberry Pi.



DAUER
etwa 90 Minuten

FÜR
UGREEN NAS (UGOS Pro) oder Raspberry Pi 4/5

STAND
Heimalarm 0.1.19 · Oktober 2026

Bevor du anfängst

Der ESP32 hängt am Schaltkontakt der Ladeschale. Schaltet der Melder bei Alarm, meldet der ESP32 das per WLAN an die Zentrale – und die gibt es an alle PCs und Handys weiter.

Einkaufsliste

- **ESP32-S3 DevKitC-1 (N16R8)** mit verlöteten Stiftleisten, ca. 10 €
  Alternativ ESP32 DevKit oder Wemos D1 mini
- 2 Jumper-Kabel Buchse–Buchse
- 2-adriges Kabel zur Ladeschale (z. B. Klingeldraht), 2 WAGO-Klemmen 221, Isolierband
- USB-C-**Daten**kabel (nicht nur Ladekabel)
- Altes USB-Netzteil für Dauerstrom
- Multimeter zum Prüfen der Ladeschale

Außerdem

- Zentrale: UGREEN NAS oder Raspberry Pi 4/5 mit Netzteil und microSD-Karte (32 GB)
- Windows-PC (für ESPHome oder Chrome/Edge)
- Android-Handy
- Zugang zum Router, WLAN-Name und -Passwort (2,4 GHz)
- Ein langes eigenes Passwort für die Zentrale

Die Schritte

1 · Ladeschale prüfen 3
1B · Kabel bauen und anschließen 4
2A · Zentrale auf der UGREEN NAS 5
2B · Raspberry Pi vorbereiten 6
2B · Zentrale auf dem Pi 7
3 · Firmware vorbereiten 8
4 · Firmware bauen und aufspielen 9
4B · Ohne Zentrale bauen: am Windows-PC 10
5 · Heimalarm am PC 11
6 · Heimalarm am Handy 12
7 · Testen 13
Die LED am ESP32 14
Pflege und Fehler 15

Schritt 1 · Ladeschale prüfen

10 min

⚠ **Erst messen, dann anschließen.** Der ESP32 verträgt keine fremde Spannung am Eingang. Bei Spannung am Kontakt: nicht anschließen, sondern einen Optokoppler dazwischen setzen. Ein einfaches Multimeter kostet im Baumarkt ca. 10–15 €.

1. **Anschluss finden:** An der Ladeschale (z. B. Motorola TPG2200) sitzt eine kleine **Steckklemme** für ein externes Gerät: zwei **runde Löcher** für die Drähte, darunter zwei **eckige Schlitze** zum Lösen.
   Draht einstecken, bis er hält. Lösen: mit einem kleinen Schraubendreher in den eckigen Schlitz drücken und den Draht herausziehen.
2. **Spannung messen:** Multimeter auf Gleichspannung (V DC). Zwischen den beiden Kontakten muss **0 V** anliegen – im Ruhezustand und bei Alarm.
3. **Durchgang messen:** Multimeter auf Durchgang. In Ruhe kein Durchgang, bei Probealarm Durchgang (Piepton).
   Ist es umgekehrt (in Ruhe Durchgang, bei Alarm offen), ist das kein Problem: Das stellst du später in der App um.

BOARD	EINGANG	MASSE	DATEI
ESP32-S3 DevKitC-1	`4` (GPIO4)	`GND`	`alarm-ich-s3.yaml`
ESP32 DevKit (klassisch)	`GPIO27`	`GND`	`alarm-ich.yaml`
Wemos D1 mini (ESP8266)	`D5` (GPIO14)	`G`	`alarm-ich-d1mini.yaml`

Kein Widerstand nötig – der ESP32 nutzt seinen eingebauten Pull-up. Die Datei aus der rechten Spalte brauchst du in Schritt 3.

✓ **Geklappt, wenn** beide Messungen wie oben aussehen.

Schritt 1B · Kabel bauen und anschließen

15 min

Ein 2-adriges Kabel führt von der Steckklemme der Ladeschale zum ESP32. Am ESP32-Ende kommen zwei Jumper-Kabel dran.

1. **Ladeschalen-Ende:** Hülle ca. 3 cm entfernen, beide Adern 8–10 mm abisolieren und fest verdrillen.
   Am besten verzinnen (mit Lötzinn), dann hält der Draht in der Steckklemme besonders gut.
2. **ESP32-Ende:** genauso abisolieren. Von zwei Jumper-Kabeln **Buchse–Buchse** jeweils eine Buchse abschneiden und 8–10 mm abisolieren.
3. **Verbinden:** jede Ader mit einem Jumper-Kabel in eine **WAGO-Klemme 221** stecken – oder fest miteinander verdrillen und mit Isolierband umwickeln. Die beiden Stellen dürfen sich nicht berühren.
4. **Am ESP32:** ein Jumper auf den **Eingang** (Tabelle Schritt 1), der andere auf **GND**. Die Reihenfolge ist egal.
   ESP32-S3: Pin `4` liegt direkt neben `RST`. Von der Ecke mit `3V3` aus zählen: 3V3, 3V3, RST, **4**. Steckt das Kabel auf RST, startet der ESP32 bei Alarm nur neu.
5. **An die Ladeschale:** die verdrillten Enden in die runden Löcher der Steckklemme stecken, bis sie halten. Kurz daran ziehen.

Keine Litze in die Jumper-Buchse stecken. Die Buchsen sind für die Stifte am ESP32 gemacht. Weiche, dünne Drähte haben darin keinen sicheren Kontakt – das führt zu verpassten Alarmen oder Fehlalarmen.

Geklappt, wenn alles fest sitzt. Prüfen kannst du das nach dem Flashen mit dem **Kontakt-Test** (Schritt 7).

Schritt 2A · Zentrale auf der UGREEN NAS

20 min

Hast du einen Raspberry Pi? Dann diese Seite überspringen und mit 2B weitermachen.

1. **Feste IP:** Im WLAN-Router (Weboberfläche oder App des Routers) die NAS in der Geräteliste suchen und ihr eine **feste IP-Adresse** geben (oft „immer gleiche IP“, „IP reservieren“ oder „DHCP-Reservierung“). IP notieren.
   Sonst kann sich die Adresse ändern, und ESP32 und App finden die Zentrale nicht mehr.
2. **Docker** im App Center installieren. In **Dateien › docker** den Ordner `heimalarm` anlegen.
3. **Dateien:** GitHub `Type-1-Diabetes/Heimalarm` → **Code** → **Download ZIP**. In `docker/heimalarm` hochladen: den Inhalt von `zentrale` und zusätzlich den Ordner `esp32`.
4. **ESPHome einschalten:** `docker-compose.yml` mit dem Editor öffnen (Windows: Rechtsklick → Öffnen mit → Editor), beim Abschnitt `esphome:` die Zeile `profiles: ["esphome"]` löschen.
   Die Docker-App der NAS beachtet Profile aus der `.env` nicht – ohne diesen Schritt gibt es keinen Container `heimalarm-esphome-1`.
5. **Passwort:** am PC `.env.example` kopieren als `env.txt`, `MQTT_PASSWORD=` ausfüllen, hochladen, auf der NAS umbenennen in `.env`.
   Sie verschwindet danach aus der Ansicht – das ist richtig.
6. **Starten:** **Docker › Projekt** → **Erstellen**, Name `heimalarm`, Pfad `docker/heimalarm`, „sofort ausführen“ → **Bereitstellen**

Geklappt, wenn im Protokoll von `heimalarm-mosquitto-1` „Adding password for user heimalarm“ steht und `heimalarm-dienst` mit `u'heimalarm'` verbunden ist.

Nur eine Compose-Datei: Liegen im Ordner `docker-compose.yml` **und** `docker-compose.yaml`, nimmt Docker nur eine davon. Die überzählige löschen. `.env` ist versteckt: in **Dateien** unten links auf das Zahnrad → „Versteckte Dateien anzeigen“.

Schritt 2B · Raspberry Pi vorbereiten

15 min

Hast du eine NAS eingerichtet? Dann 2B überspringen. Der Pi läuft danach ohne Bildschirm und Tastatur.

1. **Raspberry Pi Imager** am PC installieren (`raspberrypi.com/software`) und die microSD-Karte einstecken.
2. **Gerät:** dein Pi-Modell. **Betriebssystem:** Raspberry Pi OS (other) → Raspberry Pi OS Lite (64-bit). **Speicher:** die microSD-Karte.
3. Bei „Einstellungen anpassen?“ **Einstellungen bearbeiten**:

Hostname	`heimalarm`
Benutzer und Passwort	frei wählen und notieren
WLAN	nur, wenn kein LAN-Kabel möglich ist (Kabel ist zuverlässiger)
Dienste	SSH aktivieren, mit Passwort

4. **Schreiben**, warten, Karte in den Pi, Netzwerkkabel und Strom anschließen. 2 Minuten warten.
5. **Feste IP** im Router für das Gerät `heimalarm` einstellen (wie bei 2A) und notieren.
6. **Verbinden:** am PC Windows-Taste → **Terminal** öffnen und eingeben:

```
ssh benutzer@heimalarm.local
```

Beim ersten Mal `yes` tippen, dann das Pi-Passwort. Klappt `heimalarm.local` nicht, die IP verwenden.

Geklappt, wenn im Terminal `benutzer@heimalarm:~ $` steht.

Schritt 2B · Zentrale auf dem Raspberry Pi

15 min

Alle Befehle im Terminal aus dem letzten Schritt. Zeile für Zeile einfügen (Rechtsklick) und Enter drücken.

1 **Docker installieren** (2–5 min):

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
exit
```

Danach mit `ssh benutzer@heimalarm.local` neu verbinden.

2 **Heimalarm holen:**

```
sudo apt install -y git
git clone https://github.com/Type-1-Diabetes/Heimalarm.git
cd Heimalarm/zentrale
cp -r ../esp32 ./esp32
```

ESPHome baut aus dieser Kopie. Bei jedem Update neu kopieren (siehe „Pflege und Fehler“), sonst bleibt die Firmware alt.

3 **Passwort festlegen:**

```
cp .env.example .env
nano .env
```

Bei `MQTT_PASSWORD=` dein Passwort eintragen. Speichern: `Strg` + `O` , `Enter` , Beenden: `Strg` + `X` .

4 **Starten:**

```
docker compose up -d
docker compose logs mosquitto
```

Geklappt, wenn das Protokoll „Adding password for user heimalarm“ und „heimalarm-dienst … u'heimalarm'“ zeigt. Die Zentrale startet ab jetzt nach jedem Stromausfall von selbst.

„MQTT_PASSWORD fehlt“? `nano .env` erneut öffnen, Passwort prüfen, dann `docker compose up -d` .

Schritt 3 · Firmware vorbereiten

10 min

Die Firmware ist das Programm auf dem ESP32. Du trägst WLAN, Zentrale und deine Kennung ein – gebaut wird sie danach auf der Zentrale.

1. **ESPHome starten:** NAS: läuft schon (Schritt 2A, Zeile 4). Pi: im Terminal

```
docker compose --profile esphome up -d esphome
```

2. Im Browser `http://<IP der Zentrale>:6052` öffnen. Anmelden mit `heimalarm` und deinem `MQTT_PASSWORD`.

3. Oben rechts **Secrets** öffnen und ausfüllen:

```
wifi_ssid: "Name deines WLANs"   # 2,4 GHz
wifi_pw: "WLAN-Passwort"
ota_pw: "eigenes Update-Passwort"
mqtt_host: "IP der Zentrale"
mqtt_user: "heimalarm"
mqtt_pw: "dein MQTT_PASSWORD"
```

Save. Die Vorlage dafür ist `secrets.example.yaml`. Sicherer: eigenes Konto nur für den Melder (`MQTT_ESP_PASSWORD` in der `.env`, hier `mqtt_user: "heimalarm-esp"`), siehe Heimalarm-Update.pdf.

4. **Nur dein Board behalten:** ESPHome zeigt drei Karten, alle heißen `alarm-ich`. Unter dem Namen steht die Datei – die zu deinem Board (Tabelle Schritt 1) behalten, bei den anderen beiden ⋮ → **Löschen**.

   Sie heißen gleich wie dein ESP32. Wer aus Versehen die falsche aufspielt, legt den Melder lahm und muss ihn per Kabel neu flashen.

5. Bei deiner Karte auf **Bearbeiten**. Ganz oben steht `kennung: ich`. Für dich passt `ich` – bei weiteren Personen eine eigene Kennung wählen, z. B. `kollege`. **Save**.

Geklappt, wenn **Validieren** (Validate) im Menü der Karte (⋮) „Konfiguration ist gültig!“ zeigt.

Schritt 4 · Firmware bauen und aufspielen

15 min

1. **Bauen:** auf der Karte ⋮ → **Installieren** → **Erweiterte Optionen** → **Firmware-Binärdatei herunterladen**.
   Das dauert beim ersten Mal 5–15 Minuten (der Pi ist langsamer als die NAS).
2. **Herunterladen:** wenn fertig, **Factory format** wählen. Eine `.bin`-Datei landet im Download-Ordner.
   Die Menünamen können je nach ESPHome-Version leicht abweichen.
3. **ESP32 anstecken:** per USB-Datenkabel an den PC. Beim S3 die Buchse mit der Aufschrift **COM** bzw. **UART** nehmen.
4. **Aufspielen:** in Chrome oder Edge `web.esphome.io` → **Connect** → den USB-Anschluss wählen → **Install** → die `.bin`-Datei wählen → **Install**
5. **ESPHome wieder stoppen** – wird nur zum Bauen gebraucht.
   NAS: **Container › heimalarm-esphome-1** → Stopp. Pi: `docker compose stop esphome`

Kein Anschluss zur Auswahl? Anderes Kabel probieren (viele sind nur Ladekabel). Beim S3: Taste **BOOT** gedrückt halten, einstecken, loslassen.

ESP32 in Betrieb nehmen

6. ESP32 vom PC trennen und an das USB-Netzteil neben der Ladeschale stecken. Die Kabel zur Ladeschale bleiben dran.

Geklappt, wenn in der App (Schritt 5) auf Start die Karte **ESP32 an der Ladeschale** „online“ zeigt – mit WLAN-Stärke und IP.

Schritt 4B · Ohne Zentrale bauen: am Windows-PC

30 min

Statt Schritt 3 und 4: ESPHome direkt am PC. Gebaut und aufgespielt wird in einem Rutsch.

1. **Python installieren:** Windows-Taste, `cmd` eingeben, Enter. Dann:

```
winget install Python.Python.3.12
```

Danach das Fenster schließen und neu öffnen. Kommt bei `python --version` ein Hinweis auf den Microsoft Store: **Einstellungen › Apps › Erweiterte App-Einstellungen › App-Ausführungsaliase** → `python.exe` und `python3.exe` aus.

2. **ESPHome installieren:**

```
pip install esphome
```

Meldet `esphome version` „nicht gefunden“, überall `python -m esphome` statt `esphome` schreiben.

3. **Dateien:** GitHub → **Code** → **Download ZIP**, entpacken. Im Ordner `esp32` die `secrets.example.yaml` als `secrets.yaml` kopieren und ausfüllen (wie in Schritt 3), `ota_pw` unbedingt ändern.

4. **Eingabe im richtigen Ordner:** im Explorer den Ordner `esp32` öffnen, oben in die Adressleiste `cmd` schreiben, Enter.

5. **Bauen und aufspielen:** ESP32 an die Buchse **COM/UART**, dann

```
esphome run alarm-ich-s3.yaml
```

Das erste Mal dauert 5–15 Minuten. Bei der Frage nach dem Weg die Zeile mit `COM…` wählen. Taucht kein COM-Anschluss auf: Treiber **CH343** bzw. **CP210x** installieren oder die andere USB-Buchse probieren.

Geklappt, wenn im Log `WiFi Connected` und `MQTT Connected` stehen (beenden mit `Strg` + `C`). Spätere Updates: derselbe Befehl, dann „Over The Air“ wählen – ohne Kabel.

Schritt 5 · Heimalarm am PC

5 min pro PC

Auf jedem PC und Laptop, der bei Alarm reagieren soll. Beliebig viele.

1. **Programm laden:** `github.com/Type-1-Diabetes/Heimalarm/releases` → neueste Version → `Heimalarm_…_x64-setup.exe`

2. **Installieren:** Datei öffnen. Zeigt Windows „Der Computer wurde durch Windows geschützt“: **Weitere Informationen** → **Trotzdem ausführen**

   Das kommt, weil das Programm (noch) nicht kostenpflichtig signiert ist.

3. **Firewall:** Fragt Windows nach dem Netzwerkzugriff, **Private Netzwerke** erlauben.

4. **Betrieb und Verbindung:** **Einstellungen › Betrieb & Koppeln** → **Mit Zentrale**, dann unter **Einstellungen › Zentrale** eintragen:

Adresse	IP der Zentrale aus Schritt 2, z. B. `192.168.178.43`
Port	`9001`
Benutzer	`heimalarm`
Passwort	dein `MQTT_PASSWORD`

   → **Speichern & verbinden**

5. **Alarmquelle und Kennung:** **Einstellungen › Alarmquellen** → **Melder** einschalten. Unter **Kennung & Sperrzeit** **Meine Kennung** = `ich` – genau wie im ESP32 (`kennung`).

6. **Was der PC tun soll:** **Einstellungen › „Fährst du?“**. Standard: „Fährst du?“ mit 30 s Countdown, danach Ruhezustand.

7. **Wie laut:** im selben Bereich Ton wählen (Melder-Ton oder eigene MP3/WAV) und **Ton testen**. Heimalarm zeigt dazu die Windows-Lautstärke – auf Wunsch hebt es sie bei Alarm an.

Geklappt, wenn Start oben **Aktiv** mit grüner Pille **Verbunden** zeigt.

Gemeinsamer PC? Soll ein PC bei mehreren Personen ausgehen: **Dieser PC** → „Abfrage auch bei Alarm von“ die weiteren Kennungen eintragen. Mehrere Bildschirme zeigen die Abfrage alle gleichzeitig.

Schritt 6 · Heimalarm am Handy

5 min

Android-Handy. Es zeigt Alarme groß, gibt deine Antwort weiter und kann zusätzlich SYBOS, aPager oder Divera mitlesen. Die Einstellungen heißen am Handy **Mehr**.

1. **App laden:** am Handy `github.com/Type-1-Diabetes/Heimalarm/releases` → `Heimalarm-v….apk`
2. **Installieren:** Datei öffnen. Fragt Android nach „Unbekannte Apps installieren“: für den Browser **Zulassen**. Warnt Play Protect: **Trotzdem installieren**.
3. **Akku:** Beim ersten Start fragt die App nach der Ausnahme von der Akku-Optimierung → **Zulassen**.
   Wichtig – sonst beendet Android die App im Hintergrund.
4. **Betrieb und Verbindung:** **Mehr** › **Betrieb & Verbindung** → Mit Zentrale, dieselben Daten wie am PC → **Speichern & verbinden**
5. **Alarmquellen und Kennung:** **Mehr** › **Alarm & Quellen** → Melder an, **Meine Kennung** = `ich` – wie im ESP32 und am PC. Nutzt du SYBOS, aPager PRO oder Divera: dort ebenfalls einschalten.
6. **Mitlesen erlauben** (nur mit Alarm-App): bei der Alarm-App **Zugriff erlauben** → in der Android-Liste **Heimalarm** einschalten.
7. **Alarm groß:** **Mehr** › **Dieses Handy**. Fehlt die Erlaubnis für Vollbild-Meldungen (ab Android 14), dort **Erlauben**. Optional: eigener Alarmton mit **Ton testen**.

Geklappt, wenn Start **Aktiv · Verbunden** zeigt und am PC unter „Verbundene Apps“ jetzt auch das Handy steht.

Optional

- **Rückmeldung an Divera oder aPager:** bei der jeweiligen Quelle in **Alarm & Quellen** einschalten (Standard aus).
- **Einsatz-Logbuch und Statistik:** **Alarm & Quellen** › **Funktionen**
- **App-Sperre** mit PIN und Fingerabdruck: **Sicherheit & Backup**

Nach einem Neustart des Handys startet Heimalarm von selbst wieder.

Schritt 7 · Testen

10 min

Von harmlos bis echt. Erst wenn alle klappen, ist Heimalarm einsatzbereit.

1. **Testalarm** – auf Start **Testalarm**.
   Der PC zeigt „Fährst du?“, es passiert aber nichts. Gut zum Ausprobieren.
2. **Lautstärke** – **Ton testen** am PC und am Handy.
   Hört man es dort, wo du schläfst?
3. **Verbindungstest** – **Einstellungen** › **Test & Diagnose** → **Jetzt testen**.
   Zentrale und Zentralen-Dienst müssen grün sein. Schneller Blick: die Pille auf Start antippen.
4. **Kontakt-Test** – **Test & Diagnose** → **Test starten**, dann die Kabelenden kurz verbinden.
   Die Anzeige springt auf „geschlossen“. Es gibt keinen Alarm. Nur mit ESP32.
5. **Melder-Alarm nachspielen** – wie ein echter Melder-Alarm, geht an alle Geräte.
   Routinen laufen nur im Testmodus.
6. **Echter Probealarm** – Melder in die Ladeschale, Probealarm auslösen lassen (Leitstelle oder Wehrführung fragen).
   Die Meldung kommt in unter einer Sekunde.

Alles bereit, wenn beim Probealarm jeder PC „Fährst du?“ zeigt und das Handy den Alarm groß bringt.

Wenn etwas nicht reagiert

WAS PASSIERT	ERST PRÜFEN
Kein Gerät reagiert	Steht Start auf **Inaktiv**? Ist in **Alarm & Quellen** der Melder an? Steht überall dieselbe Kennung?
Nur ein PC reagiert nicht	Ist er verbunden (Pille auf Start)? Läuft eine Ruhezeit (**Dieser PC**)?
Handy meldet im Hintergrund nichts	Akku-Ausnahme und Vollbild-Meldungen erlaubt? **Mehr** › **Dieses Handy**
Zu leise	„Ton testen“ – Windows- bzw. Wecker-Lautstärke direkt darunter anheben.
Start: ESP32 „offline“	Strom? WLAN 2,4 GHz? `mqtt_host` und `mqtt_pw` in den Secrets richtig?
ESP32 online, Alarm kommt nicht	**Test & Diagnose** › **Kontakt-Test**: Ändert sich die Anzeige beim Verbinden der Kabelenden nicht, sitzt eine Verbindung locker. Kennung überall gleich?
Alarm kommt erst, wenn der Kontakt wieder öffnet	**Einstellungen** › **ESP32** → „Alarm, wenn der Kontakt“ auf **öffnet**.

Die LED am ESP32

Die Lampe auf dem Board zeigt, was der Melder gerade tut. Praktisch beim Einrichten und als Kontrolle an der Ladeschale.

EREIGNIS	FARBIGE LED (ESP32-S3)	EINFARBIGE LED (ESP32, D1 MINI)
Alarm (Kontakt) bis der Kontakt öffnet, höchstens 60 s	rot, schnell blinkend	schnell blinkend
Firmware-Update	türkis, sehr schnell	sehr schnell blinkend
Test, Kontakt-Test beim Kontakt-Test Dauerlicht, solange der Kontakt geschlossen ist	violett, pulsierend	pulsierend
Sucht WLAN	blau, langsam pulsierend	langsam pulsierend
Zentrale nicht erreicht was fehlt, zeigt die App auf Start	orange, blinkend	blinkend
Mit Zentrale verbunden	3 s grün, dann aus	3 s an, dann aus
Nicht verfügbar folgt der Bereitschaft auf Start	gelb, schwach	aus
Alles in Ordnung	aus (mit Ruhelicht schwach grün)	aus

Einstellen

- In der App unter **Einstellungen › Alarm & Quellen › LED am ESP32**: an/aus, Helligkeit, **Wann leuchten** (Alles · Nur Alarm und Fehler · Nur Alarm) und **Ruhelicht**. Die App erkennt, ob das Board eine farbige oder eine einfarbige LED hat, und zeigt nur, was passt.
- **Alle Farben zeigen** spielt jeden Zustand 1,5 s lang nacheinander ab.
- Die Einstellungen bleiben im ESP32 gespeichert, auch nach einem Neustart.

LED bleibt dunkel? Bei den meisten ESP32-S3-N16R8-Boards sitzt sie an `GPIO48` (so ist es eingestellt). Bei Espressif-Boards ab Version 1.1 an `GPIO38` : in `alarm-ich-s3.yaml` die Zeile `led/rgb-gpio48.yaml` in `led/rgb-gpio38.yaml` ändern und neu aufspielen.

⚠ Pflege und Fehler

Updates

- **Apps:** PC und Handy melden neue Versionen beim Start → **Installieren**
- **Zentrale auf dem Pi:**

```
cd ~/Heimalarm && git pull && cd zentrale
cp -r ../esp32/* esp32/
docker compose up -d --build
```

`cp` holt die neue ESP32-Vorlage in `zentrale/esp32` – daraus baut ESPHome. Ohne den Schritt spielt ESPHome weiter die alte Firmware auf. `secrets.yaml` bleibt erhalten; eine eigene Kennung (nicht `ich`) danach in der Board-Datei wieder eintragen.

- **Zentrale auf der NAS:** genaue Schritte in der Anleitung **Heimalarm-Update.pdf**. Kurz: Inhalt von `zentrale` und den Ordner `esp32` hochladen und vorhandene Dateien **ersetzen** (nicht `.env`, die `data`-Ordner, `secrets.yaml`), in `docker-compose.yml` wieder `profiles: ["esphome"]` löschen, Container und Image von `dienst` löschen, **Neu bereitstellen**.
- **ESP32:** nur nötig, wenn es eine neue Firmware gibt – ESPHome starten, auf deiner Karte ⋮ → **Validieren**, dann **Installieren** → **Im Netzwerk**. Ohne Kabel.

Häufige Fehler

PROBLEM	LÖSUNG
ESPHome-Seite (Port 6052) lädt nicht, Container fehlt	Container `esphome` gestartet? Auf der NAS die Zeile `profiles: ["esphome"]` gelöscht und neu bereitgestellt?
Drei Karten „alarm-ich“, alle „Online“	Normal – sie heißen gleich wie dein ESP32. Nur die Karte zu deinem Board nutzen, die anderen löschen.
Im Netzwerk: „Host not found“	Docker findet `alarm-….local` nicht. **Installieren** → **Erweiterte Optionen** → **Geräte-IP**, IP aus der App (Start → ESP32) eintragen.
Log: „mqtt took a long time“	Harmlos, kommt kurz nach dem Neustart.
Pi: App zeigt „ältere Firmware“ nach dem Flashen	Kopie in `zentrale/esp32` ist alt: `cp -r ../esp32/* esp32/`, Kennung prüfen, neu aufspielen.
Bauen bricht ab	Fehlermeldung lesen – meist fehlt ein Eintrag in den Secrets.
web.esphome.io findet den ESP32 nicht	Datenkabel, andere USB-Buchse, BOOT-Taste beim Einstecken halten.
ESP32 verbindet sich nicht mit dem WLAN	Nur 2,4 GHz. WLAN-Name genau wie im Router (Groß-/Kleinschreibung).
Protokoll: „ESP32 neu gestartet“ beim Alarm	Kabel steckt auf `RST` statt auf dem Eingang. Pins nachzählen.
Fehlalarme ohne Einsatz	**Einstellungen › Alarm & Quellen › ESP32** → Mindestdauer erhöhen.